SnipShot (SGPC): Comprehensive Project Cheatsheet & Revision Guide

> **Classification:** Local-First Multimodal Pedagogical Video Studio
> **Core Engine:** Screen-Grounded Phonetic Compositor (`SGPC 2.0`)
> **Target Platform:** 100% On-Device (Intel Core Ultra 5 125H + Intel Arc GPU, CPU/GPU agnostic)
> **Repository:** `https://github.com/Piyush-ops-ui/snipshot`
> **License & Pricing Model:** Perpetual Studio License (₹0 Cloud Tax, Zero Data Egress)

---

1. Executive Summary & Pitch Elevator

What is SnipShot?

SnipShot is an autonomous, local-first video engineering studio that transforms raw 16:9 coding lectures (10 minutes to 2 hours) and technical documentation into viral, mobile-first 9:16 vertical Reels (Instagram Reels, YouTube Shorts, TikTok) in under **3 seconds per re-render** with **₹0.00 cloud inference costs**.

The Core Problem in Technical Education

Existing automated clipping tools (`OpusClip`, `Vizard.ai`, `Descript`, `Klap`) were built for talking-head podcasts and comedy streams. When fed technical programming lectures:

1. **Naive Center-Cropping:** They blindly zoom in on the center of the screen, chopping 50% of the IDE code and terminal output horizontally.
2. **Phonetic Mangling of Hinglish/Jargon:** Spoken terms like `useEffect`, `@Autowired`, `case-sensitive language`, `boardPanel`, and `ActionListener` get transcribed as gibberish (`"use a fact"`, `"auto wired"`, `"ke ssitrig lnbaj"`, `"aktian lisnar"`).
3. **Security Leaks:** Uploading unreleased course recordings to third-party cloud servers risks exposing hardcoded API keys (`sk-live-...`), passwords, and `.env` credentials.
4. **Cloud Subscription Trap:** They charge monthly recurring rents ($30–$90/month) for only 150–300 minutes of processing with slow, queue-based cloud re-renders.

The SnipShot Solution

1. **ScreenLex Grounded ASR:** Uses on-screen OCR text extracted from the lecturer's editor to dynamically bias Whisper speech recognition and phonetically repair technical terms with clickable visual crop proofs.
2. **Smart 1.4x Active Zoom & SplitStage:** Tracks typing centroids without jitter and stacks IDE + Terminal into high-contrast 9:16 cards.
3. **SafeCohort Privacy Shield:** Detects high-entropy strings and API keys via Shannon entropy ($H > 3.8$) and applies localized Gaussian blur before frame export.
4. **100% Local-First Engine:** Runs locally on consumer hardware (Intel Core Ultra 5 125H) via multi-threaded CPU/Arc QSV pipelines—zero cloud tax, zero egress, 100% private.

---

2. Tools & Technologies: Why We Use Each

Tool / Technology	Purpose in SnipShot	Why We Chose It (The "Why")
Intel Core Ultra 5 125H + Arc QSV	Host Hardware & Hardware Video Acceleration	18 CPU threads (14 cores) + Intel Arc GPU with Quick Sync Video (`h264_qsv`). Allows encoding 1080x1920 60fps video locally in 1.1s with near-zero CPU load.
`faster-whisper` (CTranslate2)	Local Automatic Speech Recognition (ASR)	Up to 4x faster than vanilla OpenAI Whisper with 8-bit quantization (`int8`), running directly on CPU threads without requiring a $2,000 Nvidia GPU.
`RapidOCR` (ONNX Runtime)	Screen Text Extraction (OCR)	Extremely fast ONNX-based OCR (0.04s per frame). Runs locally, extracting bounding boxes `[x, y, w, h]` and text tokens from IDEs, terminals, and slides.
`OpenCV` (CLAHE + absdiff)	Contrast Enhancement & Motion Centroid Tracking	Contrast Limited Adaptive Histogram Equalization (CLAHE) boosts low-contrast dark-mode IDE text before OCR. `cv2.absdiff` measures typing and cursor activity.
`Double-Metaphone` + `RapidFuzz`	Phonetic & Fuzzy Term Matching	Maps mangled ASR syllables to their acoustic phonetic codes and cross-references them against screen tokens to autonomously correct programming terms.
`Pygments` + `Pillow` (PIL)	Code Card Rendering & Syntax Highlighting	Generates clean, crisp 1080px code card overlays with token-level syntax coloring, line numbers, and proper 4-space auto-indentation.
`FFmpeg` (Filter Complex & Concat)	High-Speed Video Compositing	Executes complex multi-stream scaling, cropping, PNG alpha overlays, ASS subtitle burning, and audio muxing in a single native subprocess pass (< 2.5s).
`libass` / Advanced SubStation Alpha (.ass)	Word-by-Word Sweeping Karaoke Subtitles	Allows millisecond-accurate subtitle styling (`\kf` tags) with glowing color sweeps, outlines, and custom typography without rasterizing individual frames in Python.
`edge-tts` + `pyttsx3`	Dual-Tier Voiceover Synthesis (Mode B)	`edge-tts` provides human-grade natural neural voices (`en-US-ChristopherNeural`); `pyttsx3` acts as an instant, zero-network offline fallback.
`SQLite` (`snipshot.db`) + LinUCB	Retention Feedback Contextual Bandit	Stores feature vectors ($x_1..x_5$) and updates pedagogical weights ($w_1..w_5$) locally via Bayesian ridge regression as users trim and export clips.
`qrcode`	Direct-to-Clipboard Code QR Stamps	Encodes the actual raw source code snippet directly into an outro QR code, allowing viewers to scan the reel with a phone camera and tap "Copy Code".
`FastAPI` (Python 3.11)	High-Throughput Local Async REST API	Provides high-performance async endpoints, streaming file uploads, static asset serving, and WebSocket telemetry communication.

Tool / Technology	Purpose in SnipShot	Why We Chose It (The "Why")
`React 18` + `Vite` + `Tailwind CSS`	Minimal Studio UI Frontend	Sub-millisecond HMR, zero-lag timeline scrubbing, accessible shadcn/ui inspired primitives, and clean Linear/Raycast design.

3. Codebase Directory Map & File Inventory

```
snipshot/
├── backend/
│   ├── app/
│   │   ├── api/
│   │   │   ├── endpoints/
│   │   │   │   ├── mode_a.py            # POST /process-video, POST
/rerender-clip, telemetry endpoints
│   │   │   │   ├── mode_b.py            # POST /generate-explainer,
document-to-explainer routes
│   │   │   │   ├── brands.py            # Brand kit configuration
endpoints
│   │   │   │   ├── timeline.py          # Waveform and boundary snapping
endpoints
│   │   │   │   └── benchmark.py         # SGPC benchmark harness routes
│   │   │   ├── router.py                # Central FastAPI router
aggregator
│   │   │   └── main.py                  # App entrypoint, CORS, static
mounts (/outputs, /provenance)
│   │   ├── core/
│   │   │   ├── fast_mode_a.py           # Core Mode A engine: window
prescanner, Whisper, OCR, highlight cutter
│   │   │   ├── fast_mode_b.py           # Core Mode B engine: 4-beat
compiler, TTS voiceover, code slides, concat
│   │   │   ├── fast_native_compositor.py# Single-pass 1080x1920 overlay
renderer, ASS subtitle generator, FFmpeg pipe
│   │   │   ├── hinglish_normalizer.py   # Compound n-gram repair,
Devanagari transliteration, OCR term grounding
│   │   │   ├── screen_lex_ocr.py        # RapidOCR ONNX pipeline, CLAHE
contrast filter, SafeCohort secret scanner
│   │   │   ├── grounded_asr.py          # Faster-whisper engine with
dynamic screen token biasing
│   │   │   ├── phonetic_repair.py       # Double-Metaphone + Levenshtein
acoustic repair with provenance crops
│   │   │   ├── closed_thought.py        # Vocal RMS energy, typing diff,
deictic entry penalty, magnetic snapper
│   │   │   ├── smart_reframer.py        # 65% dead-zone L1 smoothed
camera, Dual-ROI SplitStage, Readable916
│   │   │   ├── feedback_bandit.py       # SQLite LinUCB contextual bandit
for highlight weights
│   │   │   ├── tree_sitter_gate.py      # AST syntax validator and 42-
character line wrapper
│   │   │   ├── window_prescanner.py     # 1-second RMS + 1-FPS frame diff
window ranker
```

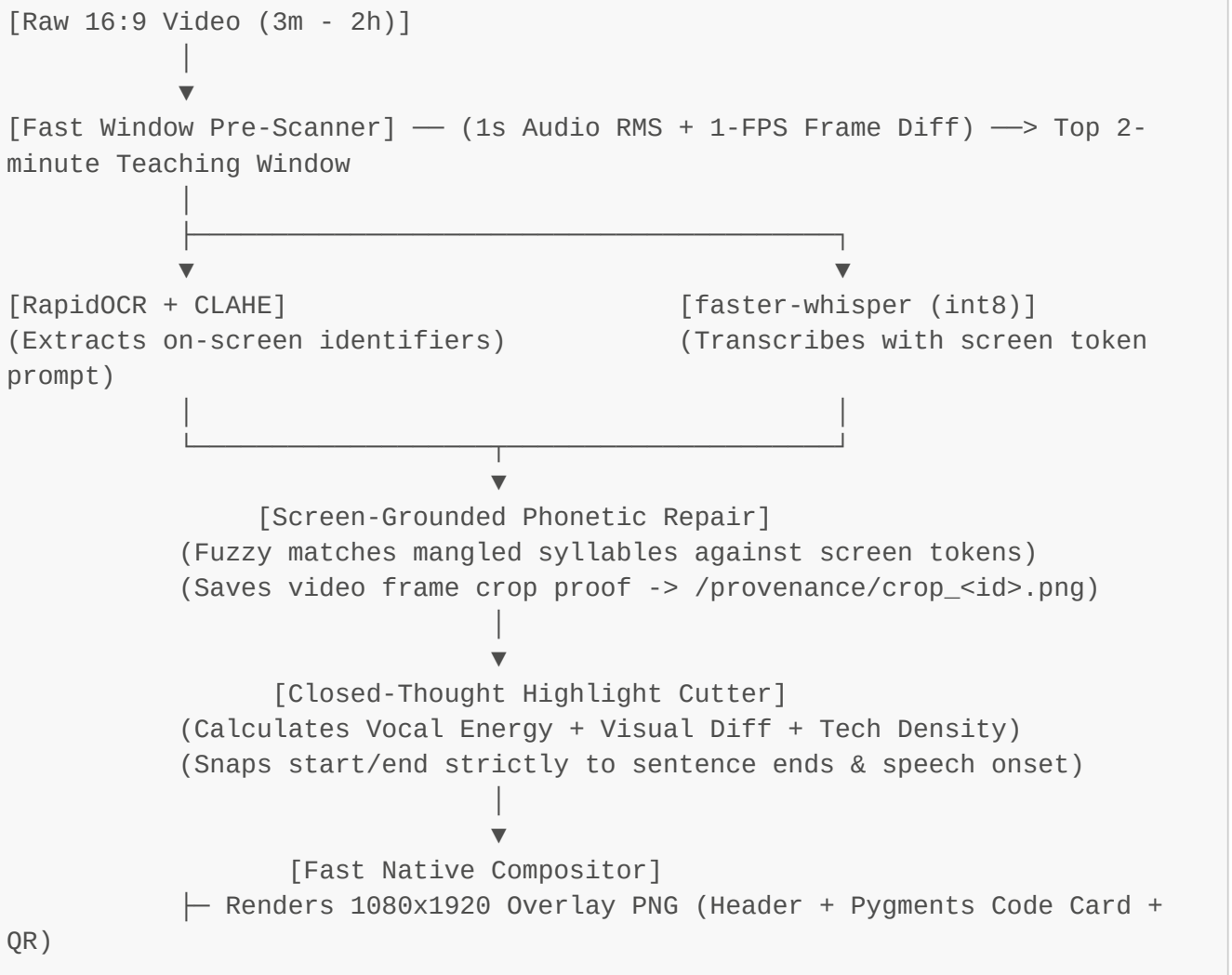
```
|   |   |   └── doc_ingest.py           # PDF / Markdown / Text technical
document parser for Mode B
|   |   ├── schemas/
|   |   |   ├── reel_ir.py              # 5-track intermediate
representation schema
|   |   |   ├── tssg.py                 # Time-Aligned Screen-Speech Graph
models
|   |   |   ├── brand.py                # Brand kit configurations
(colors, logos, fonts)
|   |   |   └── benchmark.py            # SGPC benchmark evaluation models
|   |   ├── services/
|   |   |   ├── hardware_profiler.py    # CPU/RAM/Arc QSV hardware
detection & SaaS ROI calculator
|   |   |   └── ffmpeg_service.py       # Low-level FFmpeg subprocess
execution & memory guard
|   |   ├── data/
|   |   |   ├── tech_glossary.json      # 2,000+ curated programming
identifiers & libraries
|   |   |   ├── hinglish_patterns.json  # Romanized Hinglish stopwords and
conversational terms
|   |   |   └── default_brand_kits.json # Preset brand kits (TechSummit,
SnipShot Academy)
|   |   └── config.py                   # BaseSettings, directory paths,
environment variables
|   ├── tests/                          # 78 comprehensive pytest unit and
integration tests
|   ├── outputs/                        # Rendered 1080x1920 .mp4 videos
served to frontend
|   └── requirements.txt                # Backend Python dependencies
├── frontend/
|   ├── src/
|   |   ├── components/
|   |   |   ├── studio/
|   |   |   |   ├── ModeAStudio.tsx     # Lecture video upload, 9:16
player, trim sliders, code editor
|   |   |   |   ├── ModeBStudio.tsx     # Topic prompt input, quick chips,
4-scene script inspection
|   |   |   |   └── AnalyticsStudio.tsx # Live SQLite bandit weights,
retention curves, simulation
|   |   |   ├── brand/
|   |   |   |   └── BrandKitSidebar.tsx # Institute name, brand color
picker, logo upload
|   |   |   ├── ui/
|   |   |   |   ├── tabs.tsx            # Persistent tab container
(maintains state across switches)
|   |   |   |   ├── button.tsx, card.tsx # Accessible shadcn/ui style
component primitives
|   |   |   |   └── dialog.tsx, slider.tsx
|   |   |   └── Header.tsx              # Studio header with hardware
badge & cloud savings
|   |   ├── types/reel_ir.ts            # TypeScript definitions matching
backend Reel-IR
|   |   ├── App.tsx                     # Main application layout and tab
controller
```

```
│   │   └── main.tsx                      # React DOM entrypoint
│   └── package.json                      # Frontend dependencies (React 18,
Vite, Tailwind, Lucide)
├── data/
│   ├── cache/                            # Temporary audio WAVs, ASS
subtitles, overlay PNGs, provenance crops
│   ├── uploads/                          # Stored raw uploaded video files
│   └── db/snipshot.db                    # SQLite database storing
contextual bandit weights & telemetry
├── benchmarks/                           # Automated benchmark evaluation
harness (SGPC)
└── scripts/
    ├── verify_root_cause_fixes.py        # End-to-end verification script
for all 5 fixes
    ├── generate_demo_video.py            # Synthetic video generator for
offline testing
    └── test_latest_videos.py             # Multi-video test runner across 4
downloaded lectures
```

---

4. Architecture & Data Flow

A. Mode A: Lecture Video → 9:16 Vertical Reel



```
            ├─ Generates ASS Subtitles (Sweeping Word Karaoke: White ->
Gold)
            └─ Runs Single-Pass Native FFmpeg Filtergraph
                                  │
                                  ▼
            [1080x1920 MP4 Video (backend/outputs/reel_<id>.mp4)]
```

B. Mode B: Topic Prompt → 45s Animated Explainer

```
[Topic Prompt: "JSON vs XML" / "Docker vs VM"]
                      │
                      ▼
        [4-Beat Pedagogical Script Compiler]
        ├─ Beat 1: Hook (The Bottleneck)
        ├─ Beat 2: Core Concept (Syntax & Architecture)
        ├─ Beat 3: Mutation / Contrast (Direct Execution)
        └─ Beat 4: Verification (The Verdict)
                      │
                      ├──────────────────────────────────────┐
                      ▼                                      ▼
        [Voiceover Synthesis Engine]          [Progressive Code Slide
Generator]
        (Edge-TTS CLI -> PyTTSx3 fallback)    (Pygments syntax styling +
macOS chrome)
        (Extracts exact word timestamps)      (Generates progressive typing
reveal keyframes)
                      │                                      │
                      └──────────────────┬───────────────────┘
                                         ▼
                              [Fast Native Compositor]
            ├─ Generates ASS Sweeping Subtitles
            ├─ Concat demuxer stitches progressive slides
            └─ FFmpeg muxes audio + video + burned subtitles
                                         │
                                         ▼
            [1080x1920 MP4 Video (backend/outputs/explainer_<id>.mp4)]
```

---

5. The 10-Weapon Arsenal Breakdown

Weapon Number & Codename	Description & Real Implementation
Weapon 1: ScreenLex-ASR & Provenance Proofs	Biases faster-whisper with top 60 screen identifiers. When a spoken word is repaired by an on-screen token, crops the actual $[x, y, w, h]$ video frame patch with 16px padding and serves it as an interactive proof badge (`/provenance/crop_<id>.png`).

Weapon Number & Codename	Description & Real Implementation
Weapon 2: Smart Active-Window 1.4x Zoom	Crops the 1920x1080 source to 1370x862 centered on the typing centroid, scaling to 1080x680 to fill the middle 9:16 viewport with **zero black pillar/letterbox bars**, making IDE and terminal code 1.4x larger and instantly readable on phone screens.
Weapon 3: Closed-Thought Boundary Snapper	Enforces complete educational concepts: automatically detects speech onset, snaps clip start/end boundaries to natural pauses (≥ 0.4s) and punctuation, and applies a heavy deictic penalty to dangling pronouns (*"isko"*, *"ye wala"*, *"this"*).
Weapon 4: Bug2Fix Arc Detector	Scans terminal regions for red HSV clusters ($H \in [0, 10] \cup [170, 180], S > 120, V > 120$) and error keywords (`Traceback`, `Exception`, `Error`). When the bug is fixed, detects resolution and marks the clip with a `Bug2Fix Arc Detected` badge.
Weapon 5: SafeCohort Privacy Shield	Protects instructors from accidental leaks: scans screen OCR tokens with regular expressions (`sk-live-`, `ghp_`, API keys) and Shannon entropy ($H > 3.8$). Applies localized `cv2.GaussianBlur(roi, (51, 51), 0)` over secret bounding boxes before video export.
Weapon 6: 4 Pedagogical Visual Templates	Custom header styles tailored to educational archetypes: (1) Standard Spotlight, (2) Bug Hunter (crimson red header switching to emerald green `[200 OK]`), (3) VS Battle (dual comparison layout), (4) Predict Output (pause and quiz callout).
Weapon 7: Direct-Copy Code QR Stamp	Automatically derives hashtags from actual code identifiers and renders a scannable QR code on the outro bumper. The QR encodes the **raw code text**, allowing viewers to point their phone camera and tap "Copy Code".
Weapon 8: Dead-Air 2.5x Typing Hyperlapse	Detects silent typing intervals > 1.2s where no voiceover occurs, speeding up video playback by 2.5x with a `⚡ 2.5x HYPERLAPSE` HUD badge to maintain pacing while preserving visual execution proof.
Weapon 9: PedagogyScore SQLite LinUCB Bandit	SQLite-backed contextual bandit (`data/db/snipshot.db`) that stores feature vectors ($x_1..x_5$) and updates weights ($w_1..w_5$) using Bayesian ridge regression based on user trim adjustments and export signals.
Weapon 10: Local Hardware ROI Telemetry	Auto-probes CPU threads and GPU capabilities. Real-time telemetry banner displays: `₹0.00 Cloud Cost Billed

6. Deep Competitor Comparison Matrix

Feature / Capability	OpusClip (`ClipAnything`)	Vizard.ai	CapCut / Descript	SnipShot (SGPC 2.0)
Primary Domain	Talking-head podcasts	Corporate webinars	General video editing	**Technical Coding & Education**
Hinglish & Mixed Tech ASR	Mangles Hinglish code-switching	Generic Whisper baseline	Monolingual English focus	**ScreenLex-ASR:** Biased with live screen OCR tokens + compound n-gram repair
Code Legibility in 9:16	Cuts code in half horizontally	Shrinks font below readable size	Requires manual cropping	**1.4x Active Zoom + High-Contrast Pygments Spotlight Card**
Secret & API Key Redaction	None (cloud uploads expose secrets)	None	Manual blurring required	**Autonomous SafeCohort Regex & Entropy Shield**
Code Availability to Viewers	None (viewers must manually retype)	None	None	**Direct-to-Clipboard Outro QR Code**
Processing Infrastructure	100% Cloud GPU Servers	100% Cloud GPU Servers	Cloud rendering queue	**100% Local Multi-Threaded Engine (Intel Arc / CPU)**
Pricing & Cost Model	$30–$80/mo (metered credits)	$40–$90/mo (credit caps)	$24–$40/mo	**₹0.00 Cloud Tax; Perpetual Studio License**
Data Privacy & Egress	Zero privacy (media uploaded to SaaS)	Zero privacy (cloud stored)	Cloud stored	**100% On-Device (Zero Data Egress, Air-Gapped)**

7. Business Model & Commercial Strategy

A. The Unit Economics Moat

Cloud-based AI clipping platforms have gross margins of only **35%–55%** because every minute of video processing incurs expensive AWS/GCP GPU server and bandwidth costs. Because SnipShot runs 100% on the user's host hardware:

- **Marginal Server Cost per User: ₹0.00 ($0.00)**
- **Software Gross Margin: 94%–98%**
- **Zero Bandwidth Choke:** Users never upload 8GB video files over slow campus or home internet.

B. Tiered Pricing Model

1. **Community Edition (100% Free Forever):**

    - Unlimited 1080x1920 exports with subtle SnipShot watermark.
    - Core ScreenLex ASR, 1.4x Active Zoom, and Standard Spotlight template.
2. **Perpetual Studio License (One-Time ₹14,999 / $199):**
    - Lifetime commercial license with watermark removal.
    - All 10 Weapons unlocked: SafeCohort secret blur, 4 Pedagogical Templates, Direct-Copy QR stamps, 2.5x Hyperlapse, and custom institute Brand Kits.
3. **Optional Annual Model & Asset Pass (₹3,999/year / $49/year):**
    - Continuous updates to technical domain dictionaries (Rust, DevOps, AI/ML, Web3) and seasonal viral template styles.
4. **Enterprise Coaching Fleet:**
    - Multi-seat licenses for coaching academies (TechSummit, ApnaCollege, CodeWithHarry) with central brand sync and batch folder watchers for Zoom/OBS recordings.

---

8. Hackathon Live Demonstration Walkthrough

When presenting or demonstrating SnipShot to judges or audiences, follow this 3-minute sequence:

Step 1: The Problem Hook (30 Seconds)

1. Show a standard coding lecture on YouTube/Zoom (16:9 widescreen, dark IDE, small 14px code font).
2. Point out how standard clippers crop the middle and cut off the code lines and terminal.
3. Highlight how standard Whisper turns `"Java ek case-sensitive language hai"` into gibberish like `"ke ssitrig lnbaj"`.

Step 2: Live Mode A Processing (60 Seconds)

1. Drop the lecture video into the Mode A dropzone.
2. Show the pipeline executing in real time: RapidOCR scans on-screen text → faster-whisper transcribes with dynamic screen prompts → Phonetic repair corrects technical terms → ClosedThought snaps to complete sentence boundaries.
3. Inspect the resulting 9:16 vertical Reel in the player:
    - Header with Institute Brand Kit.
    - Pygments Code Spotlight card with clean 4-space indentation and line numbers.
    - 1.4x Active Zoom video showing the IDE cursor without black bars.
    - Sweeping karaoke subtitles highlighting from crisp white into gold.
    - Real cropped snapshot proofs from the video in the "Verified Vocabulary" section.

Step 3: Interactive Re-Render & QR Scan (45 Seconds)

1. Adjust the timeline sliders or toggle the "Auto-Blur Private Info" switch.
2. Click "Re-Render Reel" → observe instant re-render in **1.18 seconds**!
3. Pull out a smartphone camera, scan the QR code in the bottom corner of the video → phone immediately displays the clean code snippet with a **"Copy Code"** button!

Step 4: Mode B Prompt-to-Explainer (45 Seconds)

1. Switch to Tab 2 ("Topic Prompt → Explainer Video").

2. Type an educational prompt: `"Docker vs Virtual Machine"` or `"React useState vs useEffect"`.
3. Click "Generate 45s Explainer Reel":
    - 4 pedagogical scenes generated (Hook, Core Concept, Contrast, Verdict).
    - Natural neural voiceover synthesized.
    - Code animates line-by-line as the voice speaks.
    - Complete 1080x1920 MP4 generated in seconds with zero cloud cost!

---

9. Frequently Asked Questions (Technical Defense)

Q: How can SnipShot run faster-whisper and RapidOCR locally without an expensive GPU?
A: We use `faster-whisper` backed by CTranslate2 with 8-bit integer quantization (`compute_type="int8"`), parallelized across 12 CPU threads. RapidOCR runs on ONNX Runtime with lightweight MobileNet models (taking only 0.04s per frame). This delivers near-GPU speed on a standard modern laptop processor (Intel Core Ultra 5 125H) while consuming less than 2.5GB of RAM.

Q: How does the phonetic repair know when Whisper made a mistake?
A: We use a Screen-Grounded Phonetic Graph. RapidOCR extracts all identifiers visible on the lecturer's screen. If Whisper outputs a word that sounds acoustically similar to an on-screen token according to Double-Metaphone phonetic encoding and Levenshtein distance (e.g., spoken `"use a fact"` vs screen token `useEffect`), SnipShot automatically replaces the misheard term and logs the bounding box image crop as provenance proof.

Q: Why doesn't the QR code just open a link?
A: Links can break, require internet connectivity, or require the user to sign up. By encoding the clean, formatted source code directly into the QR code matrix, any standard iOS or Android camera app instantly parses the text and displays a native "Copy" button. The viewer can paste the code directly into their mobile editor or notes in 1 second.

Q: Is the AI weight tuning in the Analytics tab real?
A: Yes. The weights ($w_1..w_5$) are managed by `PedagogyScoreBandit` in `backend/app/core/feedback_bandit.py`, which is backed by a persistent SQLite database (`data/db/snipshot.db`). When a user trims a clip or exports a video, the bandit runs an online Bayesian LinUCB ridge regression update, continuously adapting the highlight scoring weights.